Anuj Pratap Singh Yadav

linkedin.com/in/yadavaps77 | +91-97523-73677 | Noida, India | yadav.anuj.singh@gmail.com

Summary

Staff engineer with 11 years of building large, long-lived C++ systems at Synopsys, Microsoft and Adobe. I own the coverage subsystems of a formal-verification product end to end, lead architecture changes across team boundaries, build LLVM-based program analysis, and build AI agents that take routine work off the team.

Skills

Languages: C++ | Python | Tcl | C# | Java | TypeScript

Domain: VC Formal (DPV, FPV) | Formal verification | C++ and RTL code coverage | Cone of influence | Property density

Systems: Dependency inversion | IWYU | LLVM IR passes (dg) | Master/worker orchestration | Serialization (XML, pugixml)

Tools: Verdi | FlexNet | LSF | GDB | Linux | Perforce | Git | Win32

AI tooling: Cursor agents, skills, rules and subagents | Scheduled Jira automation

Experience

R&D Staff Engineer | Synopsys *Noida, India* **May 2023 - Present**

VC Formal (formal verification), DPV app: C++ vs RTL equivalence

- **Build-dependency decoupling (multi-org initiative):** led VC Formal's part
  - The shared C++ layer under ~15 apps and tools included engine headers, so one engine edit rebuilt every app.
  - Cut wrong-way dependencies ~73% with include-what-you-use, hide-what-you-use (forward declarations), dependency inversion through startup-injected callbacks, and relocating misplaced code.
  - Rolled out in small changes reviewed by the owning teams; in production. Presented the design in an org-wide tech talk.
- **C++ code coverage:** own it end to end
  - Shows which parts of a C++ model the formal proofs actually exercised, so C++ can be signed off like RTL.
  - Added merging across proofs and export to the standard coverage database, so C++ and RTL coverage are reviewed together in one viewer.
  - Drive changes with the viewer and GUI teams, handle customer issues, and guided its integration into a new C++ + RTL sign-off app.
- **C++ program slicer (customer-requested R&D)**
  - Building source-level cone of influence for C++ designs: the exact C++ lines a property depends on.
  - Built on the open-source dg LLVM slicer: wrote LLVM IR passes for C++ constructs it missed, and a signal-based slicing criterion that cut whole-file slices down to the lines feeding each signal.
  - Design reviewed weekly with a Distinguished Engineer.
- **Formal coverage (property density, RTL coverage):** own both
  - They report which design signals, down to the bit, and which RTL lines the properties actually check.
  - Rebuilt property density on the shared design model so DPV and FPV give the same answer, adding bit-level detail and RTL-to-RTL designs; aligned RTL coverage semantics across the two apps. In production.
- **Licensing infrastructure:** own it for the DPV app (engine side)
  - Built token licensing in the C++ proof engine on its license-checkout thread, landed behind a switch and enabled for release.
  - Moved runtime license checkout from workers to the master of the distributed solver, so one process owns keys for every tier and each key serves more workers.
  - Root-caused production checkout failures across license servers (inconsistent license-library calls, finished workers holding keys) and shipped the fixes.

- **AI agents for the team's workflow:** Cursor agents teammates use
  - A daily scheduled check that flags Jira tickets with missing or wrong validation fields and notifies the owner.
  - An agent that takes a Jira ticket, reproduces the issue, finds the root cause and drafts a fix as a code review for an engineer to check.
- **Customer escalations:** own them end to end in my areas: reproduce, root-cause, add a regression test, and ship the fix in service packs.
- **Session save/restore (serialization):** extended snapshots to capture state that was silently lost on restore (user Tcl procedures, engine problem state as XML), and kept new features' state persistent with round-trip tests.

Software Engineer II | Microsoft *Noida, India* **May 2021 - May 2023**

- **Recording Studio, PowerPoint Win32 (C++):** owned camera modes, live background blur, video export and Cameo, a camera object placed on the slide; led v1 test automation and wired Recording and Export telemetry.
- **OneDrive commerce (C#):** designed catalog filtering plus an allowlist so new commerce SKUs no longer broke OneDrive's upgrade path.
- **Stream 2.0 in Excel Win32 (C++, TypeScript):** built the video playback component for SharePoint and OneDrive video.

Software Development Engineer 2 | Adobe *Noida, India* **Aug 2015 - Apr 2021**

- **Photoshop and Premiere Elements builds:** moved from RIBS to Hyperdrive builds with parallel install bundles, cutting install time ~50% on average.
- **AEM web editor:** designed lazy loading for large files, cutting load time ~60% on files customers reported as freezing.

Education

B.Tech (Computer Science and Engineering) GPA 8.27/10	**Maulana Azad National Institute of Technology (NIT Bhopal)**	*Bhopal, MP*	**2011 - 2015**
Class 12 (CBSE) 78%	**Kendriya Vidyalaya Shivpuri**	*Shivpuri, MP*	**2009 - 2010**
Class 10 (CBSE) 85.6%	**Kendriya Vidyalaya JRC Bareilly**	*Bareilly, UP*	**2007 - 2008**

Awards

Spot Awards: Synopsys 2023, 2025, 2026 | Adobe 2020

Hobbies and Interests

Competitive programming | Cricket | Football | Chess